

Fraud Detection - Teknik Dokuman

Bu dokumanda projede ne yaptigimi anlatiyorum. Canli ortamda ne degisir onu da kısa kısa yazdim.

1 Proje Nedir?

Kaggle ieee fraud verisi kullanildi. train_transaction ve train_identity csv birlestirdim. Toplam yaklasik 590 bin satir. Fraud orani %3-4 civari yani dengesiz veri.

Accuracy kullanmadim cunku her seye non-fraud dersem %96 dogru gorunur ama hic fraud yakalamazsin. Ben PR-AUC ve recall baktim.

Akis: veri yukle -> zamana gore bol -> feature -> lightgbm -> 4 katman anomali -> context -> rule -> rag -> api

2 Veri

Veriyi buradan indirdim:

<https://www.kaggle.com/c/ieee-fraud-detection/data>

Data klasorune koydum:

- train_transaction.csv

- train_identity.csv

Test csv kullanmadim.

iki dosyayi TransactionID ile birlestirdim. Identity her islemdede yok o yuzden bazi id kolonlari bos kaliyor bu normal.

Split zamana gore yaptim rastgele degil:

Train %70, val %15, test %15

Train 413378, val 88581, test 88581

Boylece gelecek bilgi train e sizmiyor.

3 Feature Engineering

EDA notebookta baktim hangi kolonlar fraud ile ilgili diye. Ondan FE_COLS listesi cikti.

Elle ekledigim featurelar:

- D7_is_null, id_31_is_null (bos alan fraud ile iliskili edada tespit edildi)

- id_31 null orani cok yuksek (%76) o yuzden tek basina red koymadim. risk skoruna ekledim + rule R02 ile tutar birlikte

- saat, hafta sonu, log tutar
- email domain fraud oranı
- mobile mi, cihaz boş mu
- kullanıcı ortalama tutar, işlem sayısı, tutar sapması, son işlemden kaç saat

geri kalan V C D kolonları otomatik seçiliyor. null oranı %35 ten fazlaysa almıyorum çok boş kolon güvenilirmez. Bu esigi case study için pratik seçtim. Canlıda kolon bazlı null analizi, missing flag veya sklearn knn gibi doldurma yöntemleri düşünülür ve esik sabit kalmaz.

önemli: email map ve entity map sadece train de hesaplanıyor. val test e aynı map uygulanıyor sızıntı olmasın diye.

4 Model Lightgbm

Fraud etiketleri mevcut olduğu için supervised learning tercih ettim. Lgbm dengesiz ve yüksek boyutlu tabular verilerde güçlü performans gösterdi ve testte 0.44 PR-AUC elde etti. Isolation Forest anomali tespiti için faydalı olsa da etiket bilgisini kullanmadığından ana model olarak lgbm'i seçtim.

Model productionda startup aşamasında eğitilmez. MLflow üzerinden artifact olarak yüklenir Retraining süreci haftalık veya aylık periyotlarla planlanır ve data drift sürekli izlenir. Gerekirse Catboost ile ensemble yapılabilir veya labelin yetersiz olduğu senaryolarda Isolation Forest ek bir sinyal katmanı olarak kullanılabilir.

Modeli bootstrap.py içinde eğitiyorum. Final ayarlarım kabaca:

1000 tree, lr 0.008, num_leaves 128, subsample 0.8, reg var, scale_pos_weight imbalance için, early stopping average_precision ile

Final metriklerim:

val LGBM PR-AUC 0.439

test LGBM PR-AUC 0.441

fraud recall val %44.4 (1350/3042 işlem REVIEW veya BLOCK)

REVIEW içinde fraud yaklaşık %18, BLOCK içinde %20

5 Anomali 4 katman

- 1) Tutar IQR üstünde mi
- 2) Lgbm olasılık
- 3) Entity tutar sapması

4) Gece saati (EDA'da gece 2 civari peak vardi)

Bunlari agirlikla birlestirdim. Uzerine risk_boost: D7 null, id_31 null, email risk carpani.

6 Context Adjust

Guvanilir kullanicinin skorunu biraz dusuruyorum mesai saati dusuk tutar vs. false positive azalsin diye.

7 Rule Engine

config/rules.yaml 8 kural var. Oncelik sirasi var. Eslesince skora + veya - ekliyor.

onemli kurallar:

- R02 id_31 eksik + tutar 200 uzeri
- R05 gece + yuksek tutar
- R03 hizli islem + tutar sapmasi
- R01 guvenilir kullanci skor dusurur

Karar esikleri: review 0.40, block 0.75

val dagilim: PASS cogu, REVIEW 6476, BLOCK 895

8 Rag

config/fraud_policies icinde 6 txt policy dosyasi var.

sentence-transformers ile embedding yaptim. En yakin policy satirlarini cekiyorum.

ollama varsa llama ile aciklama yazdiriyor yoksa basit metin donuyor.

Trulens kullanmadim. rag testini main.py de yaptim: rule tetiklenince dogru policy dosyasi geliyor mu diye baktim hit rate %97-100 civari.

Canlida rag icin: TruLens veya RAGAS ile faithfulness olculur. vector db kullanilir. PII maskeleme lazim.

9 Api

Fastapi app/main.py

Endpoint ve Dondurdukleri

/score : skor + PASS/REVIEW/BLOCK

/explain: skor + rule metni + RAG aciklama + agent meesaj

/rules/evaluate: sadece rule

/rag/query: policy arama + custom query

Ornek body: {"transaction_id": 3400379}

Onemli: api simdilik sadece val set id kabul ediyor. val_df icinden satir ariyor

Bunu bilerek yaptim. 400 kolon json gondermek zor.

Canlida api: gercek islem json gelir. model startup ta train edilmez diskten artifact yuklenir. auth rate limit. val lookup kaldırılır.

10 Main.py

python main.py veya bash run.sh

PR-AUC recall rag hit yazar bir fraud ornegi gosterir.

11 Dosyalar

src/data_load.py src/validation.py src/features.py src/bootstrap.py

src/anomaly_engine.py src/scores.py src/context_adjust.py

src/rule_engine.py src/rag_pipeline.py src/agents.py

main.py app/main.py eda/eda.ipynb run.sh run_api.sh

12 Calistirma

pip install -r requirements.txt

data ya csv koy -> python main.py

api: bash run_api.sh port 8001

13 Canlida Ne Farkli?

Canlida veri surekli akar csv degildir. Entity ve velocity icin online feature store lazim. Model startupta egitilmez kayitli artifact yuklenir retrain ve drift izlenir. Rule'lar admin panelden yönetilir REVIEW islemleri case ekranina gider. RAG Trulens ile test edilir policy vector db'de tutulur. API val id yerine gercek json alır auth vardır.